

Componentes principales sobre los predictores diarios

Bloque 6a — MA2003B, equipo 7

Tabla de contenidos

0. Carga y bloque de predictores	1
1. Los faltantes no son aleatorios	2
2. Partición y re-estandarización	5
3. Descomposición y criterio de retención	5
4. Cargas y nombre de los ejes	8
5. Puntuaciones para #55	12
6. Cierre	13

Las componentes principales no son un objetivo del proyecto: son el paso que arregla la multicolinealidad antes de que los predictores entren al modelo de clasificación de #55. NO2 y NOX correlacionan a 0.89 y PM10 con PM2.5 a 0.62, de modo que no pueden tratarse como coordenadas independientes en una regresión logística.

Su producto real son **ejes con nombre**. Lo que se espera encontrar, sin forzarlo, es un eje fotoquímico (TOUT+, SR+, RH−), uno de combustión primaria y otro de estancamiento (WSR baja, PRS alta). Que aparezcan o no es el hallazgo.

De ahí la regla que ordena todo el documento: **el eje de estancamiento no se construye a mano**. Armar un índice con viento bajo y presión alta y presentarlo después como resultado sería imponer el supuesto y llamarlo evidencia. Si esas variables co-varían, la descomposición lo encuentra sola.

Opera sobre `sima_transformado_diario.parquet`, la salida de #52. Las figuras se escriben en `reports/figuras/` y las puntuaciones en `data/processed/`, para que #55 las consuma sin recalcular el ajuste.

0. Carga y bloque de predictores

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

RAIZ = Path.cwd().parent
```

```

PROC = RAIZ / "data" / "processed"
FIGS = RAIZ / "reports" / "figuras"
FIGS.mkdir(parents=True, exist_ok=True)

plt.rcParams.update({"font.size": 9, "font.family": "serif",
                    "axes.spines.top": False, "axes.spines.right": False,
                    "figure.dpi": 150})
AZUL, ROJO, GRIS, CLARO = "#3b6ea5", "#b5443a", "#8c8c8c", "#d9d9d9"

D = pd.read_parquet(PROC / "sima_transformado_diario.parquet")
D["anio"] = D.fecha.dt.year
print(f"diario transformado: {D.shape}")

```

diario transformado: (23931, 34)

El bloque que entra a la descomposición se define por exclusión, y cada exclusión tiene un motivo distinto.

Las categóricas no entran. zona, tipo_dia, temporada, festivo, fin_de_semana y llovio son binarias o de pocos niveles; una columna con dos valores no tiene una dirección de máxima varianza que signifique algo, y mezclarla con variables continuas infla artificialmente la varianza explicada. Entran directo al modelo de #55, sin pasar por aquí.

Las derivadas de 03 tampoco. MDA8, 03_max_1h y las indicatoras excede_* son la respuesta. Meterlas como predictores sería fuga de información: el PCA construiría ejes que ya contienen lo que el clasificador debe predecir.

```

RESPUESTA = ["MDA8", "03_max_1h", "excede_anio_1", "excede_anio_3",
             "excede_anio_5", "excede_1h"]
CATEGORICAS = ["zona", "tipo_dia", "temporada", "festivo", "fin_de_semana",
               "llovio"]
IDENTIFICA = ["estacion", "fecha", "anio", "mes", "msnm", "ventanas_validas"]

CONTINUAS = [c for c in D.columns
              if c not in RESPUESTA + CATEGORICAS + IDENTIFICA]
print(f"{len(CONTINUAS)} continuas candidatas:\n{CONTINUAS}")

```

16 continuas candidatas:

['TOUT_max', 'SR_acum', 'NO_06_09', 'NO2_06_09', 'NOX_06_09', 'CO_06_09', 'WSR_media', 'WSR_min']

1. Los faltantes no son aleatorios

El PCA necesita filas completas, así que conviene medir qué cuesta exigir las antes de exigir las.

```
falt = pd.DataFrame({
    "faltantes": D[CONTINUAS].isna().sum(),
    "pct": D[CONTINUAS].isna().mean() * 100,
}).sort_values("faltantes", ascending=False)
print(falt.round(1).to_string())
print(f"\nfilas completas en las {len(CONTINUAS)}: "
      f"{D[CONTINUAS].notna().all(axis=1).sum():,} de {len(D):,}")
```

	faltantes	pct
PM2.5_media	5229	21.9
RH_media	2067	8.6
RH_min_diurna	1995	8.3
SO2_media	1720	7.2
NO2_06_09	1624	6.8
NO_06_09	1598	6.7
NOX_06_09	1584	6.6
CO_06_09	1554	6.5
viento_u_media	1387	5.8
viento_v_media	1387	5.8
TOUT_max	1272	5.3
SR_acum	1018	4.3
WSR_media	959	4.0
WSR_min	959	4.0
PM10_media	805	3.4
PRS_media	751	3.1

filas completas en las 16: 14,719 de 23,931

PM2.5_media domina la lista. Pero el volumen no es el problema: el problema es dónde faltan.

```
disp = (D.groupby("estacion", observed=True)["PM2.5_media"]
        .apply(lambda s: s.notna().mean()).sort_values())
print("disponibilidad de PM2.5 por estación:")
print(disp.round(3).to_string())
```

disponibilidad de PM2.5 por estación:

estacion	
NE3	0.000
NO3	0.101
NO	0.540
SUR	0.789
NE2	0.790
NO2	0.875
SE	0.878
CE	0.882

SO	0.902
NTE	0.909
SE2	0.940
SO2	0.945
NE	0.951
SE3	0.954
NTE2	0.978

No son días perdidos al azar: **NE3 no tiene un solo registro y NO3 apenas 12.5 %**. Son dos estaciones sin monitor de PM2.5. Exigir la variable completa no descarta observaciones, descarta estaciones enteras — y con ellas, la representación espacial que #55 necesita para sus indicadores de zona.

```
SIN = "PM2.5_media"
c16 = D[CONTINUAS].notna().all(axis=1)
c15 = D[[c for c in CONTINUAS if c != SIN]].notna().all(axis=1)

ret = pd.DataFrame({"total": D.groupby("zona", observed=True).size(),
                    "con_PM25": D[c16].groupby("zona", observed=True).size(),
                    "sin_PM25": D[c15].groupby("zona", observed=True).size()})
ret["ret_con"] = ret.con_PM25 / ret.total
ret["ret_sin"] = ret.sin_PM25 / ret.total
print(ret.round(3).to_string())
```

	total	con_PM25	sin_PM25	ret_con	ret_sin
zona					
Centro	1642	1332	1488	0.811	0.906
Noreste	4926	2100	3152	0.426	0.640
Noroeste	4227	1656	2402	0.392	0.568
Norte	3284	1978	2072	0.602	0.631
Sur	1642	1112	1346	0.677	0.820
Sureste	4926	3877	4111	0.787	0.835
Suroeste	3284	2664	2777	0.811	0.846

El castigo cae sobre el Noroeste y el Noreste, y el Noroeste es justamente la zona de mayor tasa de excedencia (41.4 %, docs/05_estacionalidad_ozono.pdf). Entrenaríamos el clasificador con la zona de más riesgo a media representación, a cambio de una variable de aerosoles que no es precursor directo de ozono y cuya señal ya aporta PM10 media, correlacionada con ella a 0.62.

Decisión: PM2.5_media se excluye del bloque de predictores. No se imputa, porque en NE3 no hay nada que imputar: rellenar el 100 % de una estación sería inventar la variable, no completar huecos.

```
PRED = [c for c in CONTINUAS if c != SIN]
print(f"{len(PRED)} predictores: {PRED}")
```

```
15 predictores: ['TOUT_max', 'SR_acum', 'NO_06_09', 'NO2_06_09', 'NOX_06_09', 'CO_06_09', 'WSR
```

2. Partición y re-estandarización

El PCA se ajusta **solo con 2021–2024** y se aplica a 2025. Ajustarlo sobre todo el periodo filtraría información del conjunto de validación hacia la construcción de los ejes.

Hay un detalle que corregir aquí. Las continuas ya vienen estandarizadas de #52, pero la media y la desviación se estimaron sobre **todo** el periodo: sobre las 23,931 filas dan exactamente 0 y 1, y restringidas a entrenamiento no. Como el PCA sobre la matriz de correlación es sensible al centrado y la escala, el bloque se re-estandariza con los momentos de entrenamiento.

```
es_tr = D.anio.between(2021, 2024)
completa = D[PRED].notna().all(axis=1)

TR = D[es_tr & completa]
VA = D[~es_tr & completa]
print(f"ajuste 2021-2024: {len(TR):,} filas      aplicación 2025: {len(VA):,}
      ↪  filas")

mu, sd = TR[PRED].mean(), TR[PRED].std(ddof=0)
Xtr = ((TR[PRED] - mu) / sd).to_numpy()
Xva = ((VA[PRED] - mu) / sd).to_numpy() # momentos de train, nunca de 2025

print(f"control train - |media| máx {np.abs(Xtr.mean(0)).max():.1e}, "
      f"|sd-1| máx {np.abs(Xtr.std(0, ddof=0) - 1).max():.1e}")
```

```
ajuste 2021-2024: 16,008 filas      aplicación 2025: 1,340 filas
control train - |media| máx 4.6e-17, |sd-1| máx 2.2e-16
```

Queda una fuga que no se puede deshacer desde aquí y que se declara como limitación: el de Box-Cox de #52 se estimó con el periodo completo. Es fuga no supervisada —afecta la forma marginal de cada variable, nunca la respuesta— y revertirla exigiría rehacer el dataset transformado que #52 ya entregó y que #55 consume.

3. Descomposición y criterio de retención

El criterio se fijó **antes** de correr el análisis, en el texto de la issue #54: **método del codo** sobre el scree plot. La varianza acumulada y el criterio de Kaiser se reportan como contraste, no para cambiar la decisión.

```
R = np.corrcoef(Xtr, rowvar=False)
lam, V = np.linalg.eigh(R)
orden = lam.argsort()[::-1]
lam, V = lam[orden], V[:, orden]

p = len(PRED)
scree = pd.DataFrame({"eigenvalor": lam,
```

```

        "prop": lam / p,
        "acumulada": (lam / p).cumsum()},
        index=[f"PC{i+1}" for i in range(p)])
print(scree.round(4).to_string())
print(f"\nKaiser (>1): {(lam > 1).sum()} componentes, "
      f"{(lam[lam > 1] / p).sum():.1%} de varianza")

```

	eigenvalor	prop	acumulada
PC1	3.9147	0.2610	0.2610
PC2	2.9728	0.1982	0.4592
PC3	1.5028	0.1002	0.5594
PC4	1.1658	0.0777	0.6371
PC5	1.0875	0.0725	0.7096
PC6	0.9258	0.0617	0.7713
PC7	0.7472	0.0498	0.8211
PC8	0.6064	0.0404	0.8615
PC9	0.5762	0.0384	0.8999
PC10	0.4890	0.0326	0.9325
PC11	0.4241	0.0283	0.9608
PC12	0.2640	0.0176	0.9784
PC13	0.2428	0.0162	0.9946
PC14	0.0676	0.0045	0.9991
PC15	0.0133	0.0009	1.0000

Kaiser (>1): 5 componentes, 71.0% de varianza

Kaiser retendría cinco componentes. No decide nada aquí —se reporta como contraste, según el checklist— pero conviene notar por qué es más generoso: > 1 es un umbral fijo que no mira la forma de la curva, solo si una componente explica más que una variable original.

El código sí mira la forma, y admite dos formalizaciones que en estos datos no coinciden. Se calculan ambas para no elegir la conveniente después.

```

x = np.arange(1, p + 1)
# (a) distancia de cada punto a la recta que une los extremos del scree
x1, y1, x2, y2 = x[0], lam[0], x[-1], lam[-1]
dist = np.abs((y2 - y1) * x - (x2 - x1) * lam + x2 * y1 - y2 * x1) / np.hypot(y2 -
    ↪ y1, x2 - x1)
# (b) curvatura: segunda diferencia del eigenvalor
curv = np.diff(lam, 2)

print("distancia a la recta extremo-a-extremo:")
print(pd.Series(dist.round(3), index=scree.index).head(6).to_string())
print("\ncurvatura:")
print(pd.Series(curv.round(3), index=scree.index[1:-1]).head(5).to_string())
print("\ncaídas sucesivas:", np.round(-np.diff(lam), 3)[:6])

```

distancia a la recta extremo-a-extremo:

PC1	0.000
PC2	0.639
PC3	1.786
PC4	1.843
PC5	1.650
PC6	1.537

curvatura:

PC2	-0.528
PC3	1.133
PC4	0.259
PC5	-0.083
PC6	-0.017

caídas sucesivas: [0.942 1.47 0.337 0.078 0.162 0.179]

La distancia máxima cae en PC4 (1.843) pero PC3 queda a 1.786: un margen de 3 %, que no decide. La curvatura sí es concluyente —1.133 en PC3 contra 0.259 en PC4, cuatro veces— y es además la lectura literal de «codo»: el punto donde la curva se dobla. Las caídas sucesivas lo confirman; el tramo empinado (0.94, 1.47) termina en PC3 y a partir de ahí la pendiente se aplanan (0.34, 0.08).

Se retienen tres componentes, 55.9 % de la varianza.

K = 3

```
fig, ax = plt.subplots(figsize=(3.4, 2.5))
ax.plot(x, lam, marker="o", ms=4, lw=1.4, color=AZUL, zorder=3)
ax.plot(x[:K], lam[:K], marker="o", ms=5, lw=0, color=ROJO, zorder=4)
ax.axhline(1, lw=.8, ls=":", color=GRIS, zorder=2)
ax.annotate("Kaiser (=1)", xy=(p, 1), xytext=(p - 0.3, 1.25),
            ha="right", fontsize=6.5, color=GRIS)
ax.annotate("codo", xy=(K, lam[K - 1]), xytext=(K + 1.6, lam[K - 1] + .75),
            fontsize=7, color=ROJO,
            arrowprops=dict(arrowstyle="->", lw=.8, color=ROJO))
ax.set_xlabel("Componente", fontsize=8)
ax.set_ylabel("Eigenvalor", fontsize=8)
ax.set_xticks(x); ax.tick_params(labelsize=6.5)
ax.grid(axis="y", lw=.4, color=CLARO, zorder=0); ax.set_axisbelow(True)
fig.tight_layout()
fig.savefig(FIGS / "pca_scree.pdf", bbox_inches="tight"); plt.show()
```

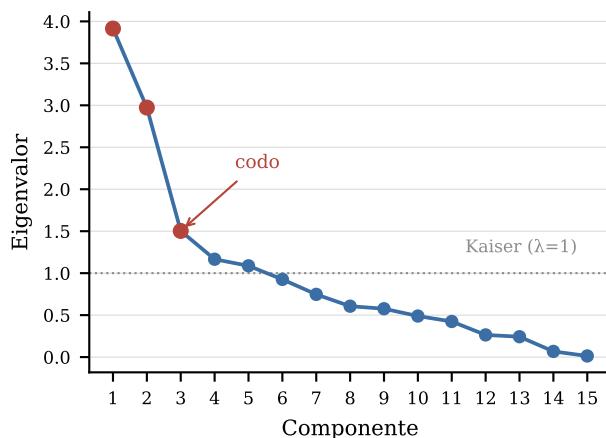


Figura 1: Scree plot de los 15 predictores. El codo por curvatura cae en PC3.

4. Cargas y nombre de los ejes

Las cargas son el eigenvector escalado por la raíz del eigenvalor, de modo que se leen como la correlación entre la variable y la componente.

El signo de un eigenvector es arbitrario: `numpy` puede devolver la dirección opuesta en otra máquina y la interpretación se invertiría. Se fija con un ancla declarada por componente —NOX, TOUT_max y PRS_media— elegida por ser la variable que da nombre al eje, no por el resultado que produce.

```
ANCLA = {0: "NOX_06_09", 1: "TOUT_max", 2: "PRS_media"}
```

```
L = V[:, :K] * np.sqrt(lam[:K])
for j, var_ancla in ANCLA.items():
    if L[PRED.index(var_ancla), j] < 0:
        L[:, j] *= -1
        V[:, j] *= -1
```

```
NOMBRES = ["Carga primaria de combustión",
            "Forzamiento fotoquímico",
            "Estancamiento atmosférico"]
```

```
cargas = pd.DataFrame(L, index=PRED, columns=NOMBRES)
print(cargas.round(3).to_string())
```

	Carga primaria de combustión	Forzamiento fotoquímico	Estancamiento atmosférico
TOUT_max	-0.075	0.717	0.22
SR_acum	0.014	0.652	0.14
NO_06_09	0.821	-0.072	-
0.291			
NO2_06_09	0.813	-0.082	-
0.286			

NOX_06_09 0.315	0.884	-0.104	-
CO_06_09 0.258	0.475	0.099	-
WSR_media 0.407	-0.446	0.602	-
WSR_min 0.500	-0.481	0.398	-
PRS_media RH_media 0.301	-0.141 -0.402	-0.340 -0.701	0.39 -
RH_min_diurna 0.368	-0.392	-0.774	-
viento_u_media viento_v_media 0.262	0.417 -0.184	-0.396 0.153	0.48 -
PM10_media SO2_media 0.006	0.659 0.400	0.245 0.186	0.01 -

```
for j, nom in enumerate(NOMBRES):
    s = cargas.iloc[:, j].sort_values(key=np.abs, ascending=False).head(5)
    print(f"{nom} ({lam[j] / p:.1%}): " +
          ", ".join(f"{v}-{c:+.2f}" for v, c in s.items()))
```

Carga primaria de combustión (26.1%): NOX_06_09+0.88, NO_06_09+0.82, NO2_06_09+0.81, PM10_media+0.48

Forzamiento fotoquímico (19.8%): RH_min_diurna-0.77, TOUT_max+0.72, RH_media-0.70, SR_acum+0.65, WSR_media+0.60

Estancamiento atmosférico (10.0%): WSR_min-0.50, viento_u_media+0.48, WSR_media-0.41, PRS_media+0.39, RH_min_diurna-0.37

Carga primaria de combustión (26.1 %). NOX +0.88, NO +0.82, NO2 +0.81, PM10 +0.66, CO +0.48, SO2 +0.40, con el viento en contra (WSR_min -0.48). Es la emisión vehicular de la ventana matutina acumulándose: los precursores suben juntos y lo hacen más cuando el aire no los dispersa.

Forzamiento fotoquímico (19.8 %). TOUT_max +0.72 y SR_acum +0.65 contra RH_min_diurna -0.77 y RH_media -0.70. Días calurosos, soleados y secos: la firma TOUT+, SR+, RH- que se anticipaba, encontrada sin imponerla.

Estancamiento atmosférico (10.0 %). WSR_min -0.50, WSR_media -0.41, PRS_media +0.39, RH_min_diurna -0.37. Viento débil con presión alta, el patrón anticiclónico. Este es el eje que la issue prohibía construir a mano, y apareció solo: nadie le indicó a la descomposición que viento bajo y presión alta van juntos.

Nota de alcance, la misma que impone #55: los tres ejes se describen como **consistentes con** combustión, fotoquímica y estancamiento. Sin mediciones de compuestos orgánicos volátiles no hay base para afirmar un mecanismo ni para atribuir la formación de ozono a un régimen NOx o COV.

```

rng = np.random.default_rng(0)
sub = rng.choice(len(Xtr), size=min(3000, len(Xtr)), replace=False)
Z = Xtr @ V[:, :K]

PAL = {"invierno": "#3b6ea5", "primavera": "#4f9d69",
       "verano": "#b5443a", "otoño": "#c98b28"}
temp = TR["temporada"].astype(str).to_numpy()[sub]

# Los nombres completos no caben: `viento_u_media` mide 14 caracteres y la
# figura ocupa dos columnas del reporte. Se abrevian y el pie los traduce.
CORTO = {"TOUT_max": "TOUT", "SR_acum": "SR", "NO_06_09": "NO",
         "NO2_06_09": "NO2", "NOX_06_09": "NOX", "CO_06_09": "CO",
         "WSR_media": "WSR", "WSR_min": "WSR mín", "PRS_media": "PRS",
         "RH_media": "RH", "RH_min_diurna": "RH mín",
         "viento_u_media": "viento u", "viento_v_media": "viento v",
         "PM10_media": "PM10", "SO2_media": "SO2"}
FS = 5.2

def coloca(tips, textos, semi, r0=1.12, paso=.07, tope=3.0):
    """Aleja radialmente cada rótulo hasta que su caja no toque las ya puestas.

    La separación no puede ser un radio: un rótulo de nueve caracteres es unas
    cinco veces más ancho que alto, y varias cargas apuntan casi en la misma
    dirección -`NO`-, -`NO2`- y -`NOX`- son el caso extremo-. Se compara caja contra
    caja, de la flecha más larga a la más corta.
    """
    puestas, salida = [], {}
    for i in sorted(range(len(tips)), key=lambda j: -np.hypot(*tips[j])):
        d = np.hypot(*tips[i])
        if d == 0:
            continue
        u, r, (hx, hy) = np.array(tips[i]) / d, r0, semi(textos[i])
        while r < tope:
            c = u * d * r
            if not any(abs(c[0] - q[0]) < hx + qx and abs(c[1] - q[1]) < hy + qy
                       for q, qx, qy in puestas):
                break
            r += paso
        puestas.append((u * d * r, hx, hy))
        salida[i] = u * d * r
    return salida

fig, axes = plt.subplots(1, 2, figsize=(6.3, 3.3))
planos = [(0, 1), (0, 2)]

for ax, (a, b) in zip(axes, planos):

```

```

for t, c in PAL.items():
    m = temp == t
    ax.scatter(Z[sub][m, a], Z[sub][m, b], s=1.8, alpha=.20,
               color=c, linewidths=0, label=t, zorder=2)
# Límites por percentil: unos pocos días extremos estiraban el eje de
# estancamiento hasta -10 y aplastaban la nube contra el centro.
lim = np.abs(np.percentile(Z[sub][:, [a, b]], [.5, 99.5], axis=0)).max() *
→ 1.32
ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim)
ax.axhline(0, lw=.4, color=CLARO, zorder=1)
ax.axvline(0, lw=.4, color=CLARO, zorder=1)
ax.set_xlabel(f"PC{a+1} - {NOMBRES[a]}", fontsize=7)
ax.set_ylabel(f"PC{b+1} - {NOMBRES[b]}", fontsize=7)
ax.tick_params(labelsize=6)

# Las cajas se miden en unidades de datos, así que hace falta el tamaño
# definitivo de los ejes: tight_layout primero, flechas después.
fig.tight_layout(rect=(0, .05, 1, 1))

for ax, (a, b) in zip(axes, planos):
    ancho_pulg, alto_pulg = (fig.get_size_inches() * ax.get_position().size)
    lim = ax.get_xlim()[1]
    ux, uy = 2 * lim / ancho_pulg, 2 * lim / alto_pulg      # datos por pulgada

    def semi(t, ux=ux, uy=uy):
        return (len(t) * .58 * FS / 72 / 2 * ux, 1.45 * FS / 72 / 2 * uy)

    largo = np.hypot(L[:, a], L[:, b])
    vis = [i for i in range(len(PRED)) if largo[i] >= .30]    # cortas: ilegibles
    tips = [(L[i, a] * lim / 1.32, L[i, b] * lim / 1.32) for i in vis]
    txt = [CORTO[PRED[i]] for i in vis]
    pos = coloca(tips, txt, semi)

    for j, i in enumerate(vis):
        dx, dy = tips[j]
        ax.arrow(0, 0, dx, dy, color="#1a1a1a", lw=.65, alpha=.9,
                 head_width=lim * .032, length_includes_head=True, zorder=4)
        ax.annotate(txt[j], pos[j], fontsize=FS, color="#1a1a1a",
                    ha="center", va="center", zorder=6,
                    bbox=dict(boxstyle="square,pad=.12", fc="white",
                              ec="none", alpha=.8))

axes[0].legend(fontsize=6, markerscale=3.5, frameon=False, ncols=4,
               loc="lower center", bbox_to_anchor=(1.09, -.28),
               columnspacing=1.2, handletextpad=.2)
fig.savefig(FIGS / "pca_biplot.pdf", bbox_inches="tight"); plt.show()

```

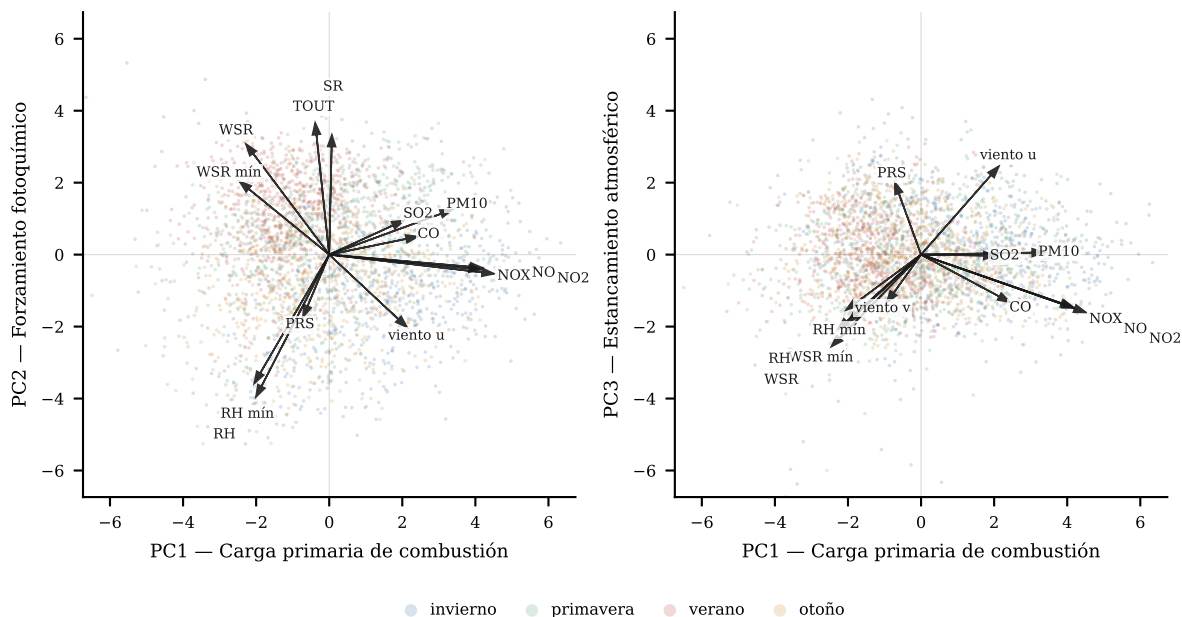


Figura 2: Biplot de los ejes retenidos. Puntos: submuestra de 3,000 observaciones de entrenamiento por temporada. Flechas: cargas escaladas al marco. Nombres abreviados: TOUT y SR son máximo y acumulado diarios; NO, NO2, NOX y CO son la ventana 06–09 h; el resto son medias diarias, salvo WSR mín y RH mín.

5. Puntuaciones para #55

Las puntuaciones se calculan proyectando sobre los eigenvectores de entrenamiento. Validación usa esos mismos vectores y los momentos de entrenamiento: 2025 se transforma, nunca se ajusta.

```
def puntuar(bloque, X):
    S = pd.DataFrame(X @ V[:, :K], index=bloque.index,
                     columns=[f"PC{i+1}" for i in range(K)])
    S.insert(0, "particion", np.where(bloque.anio <= 2024, "entrenamiento",
                                      "validacion"))
    return pd.concat([bloque[["estacion", "fecha"]], S], axis=1)

PUNT = pd.concat([puntuar(TR, Xtr), puntuar(VA, Xva)]).sort_values(
    ["fecha", "estacion"]).reset_index(drop=True)

print(PUNT.groupby("particion")[["PC1", "PC2", "PC3"]
    .agg(["mean", "std"]).round(3).to_string())
```

	PC1		PC2		PC3	
	mean	std	mean	std	mean	std
particion						
entrenamiento	0.000	1.979	0.000	1.724	0.000	1.226
validacion	-0.118	2.189	0.054	1.553	0.233	1.202

Las medias de validación no son cero, y no deberían serlo: 2025 termina el 30 de junio, así que le sobra primavera y le falta otoño. El desplazamiento es el calendario, no un error de ajuste; es la misma limitación estacional que #55 declara para su partición.

```
PUNT.to_parquet(PROC / "pca_puntuaciones_diarias.parquet", index=False)
cargas.to_csv(PROC / "pca_cargas.csv")
print(f"guardadas {len(PUNT):,} puntuaciones y {cargas.shape} cargas")
```

guardadas 17,348 puntuaciones y (15, 3) cargas

6. Cierre

Tres componentes retienen el 55.9% de la varianza de quince predictores correlacionados y admiten lectura física completa. La correlación de 0.89 entre NO2 y NOX que motivaba el análisis queda absorbida en un solo eje, y las tres coordenadas que hereda #55 son ortogonales por construcción.

Kaiser habría retenido cinco (71.0%) y una lectura alternativa del codo, cuatro (63.7%). Ninguna de las dos se adoptó, porque el criterio quedó comprometido antes de ver el resultado y la curvatura del scree plot es concluyente en PC3. Queda registrado para que el lector juzgue el margen.